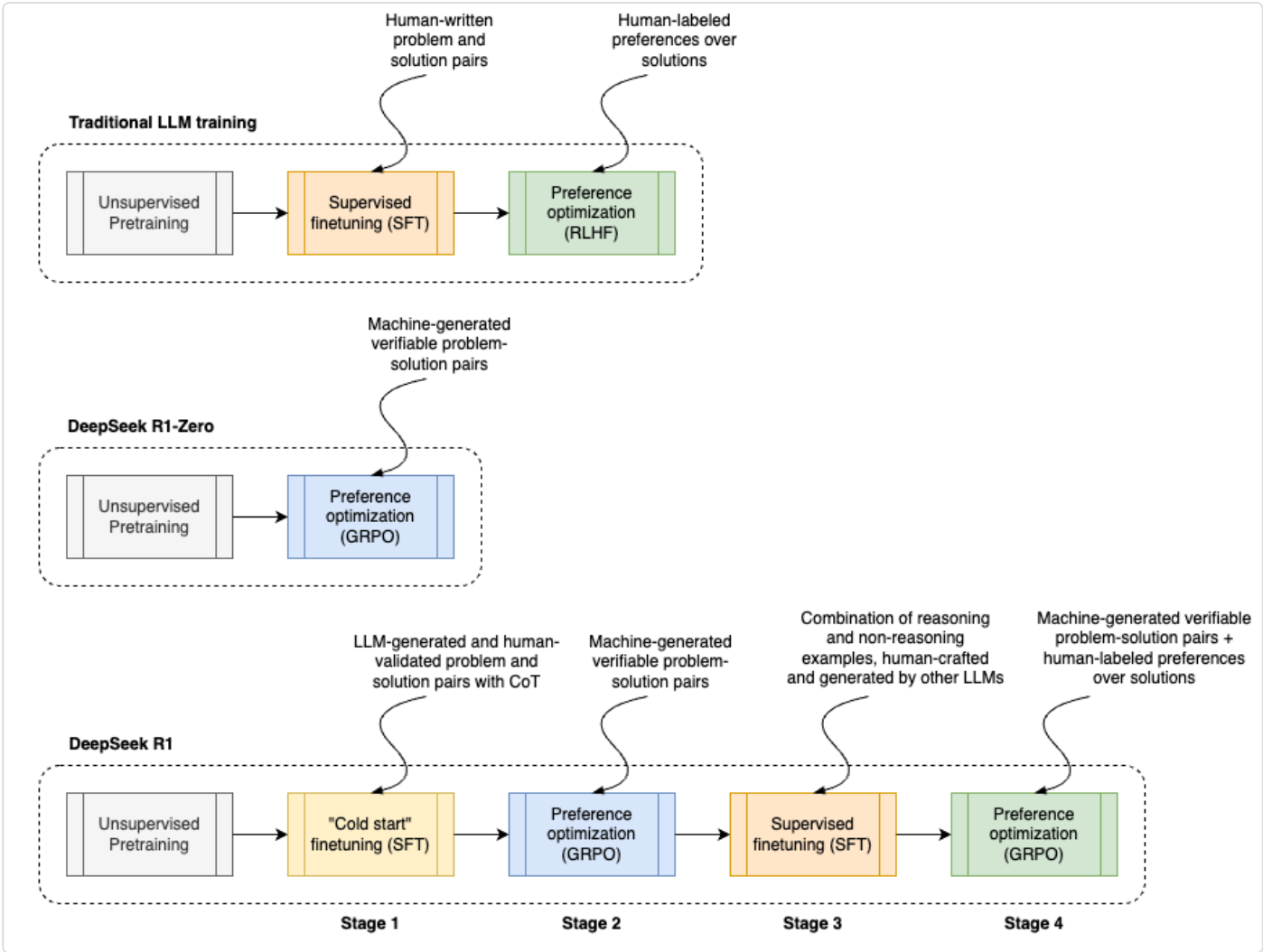


DeepSeek R1 and R1-Zero Explained



(../img/DeepSeek-R1.drawio.png)

Why DeepSeek R1 is a major innovation and a big deal?

DeepSeek's R1 addressed a major AI challenge: teaching models to reason step-by-step without relying on large datasets of labeled reasoning examples. Their results, demonstrated in the R1-Zero experiment, show that reinforcement learning alone can develop advanced reasoning skills, avoiding the need for extensive supervised training.

R1-Zero learned to produce detailed reasoning steps, check its own solutions, and spend more computation time on harder problems. Remarkably, these behaviors were not programmed explicitly but emerged during training. This demonstrates that with careful design, reinforcement learning can create models that reason effectively and adapt flexibly.

[抛弃的基于模型打分的reward model](#)

A key breakthrough was in the reward system. Instead of complex, multi-network systems prone to "reward hacking," DeepSeek used a simpler, rule-based approach. The model was rewarded for generating correct answers—validated automatically—and for producing clear, structured reasoning. This approach ensures not only accurate problem-solving but also consistent and readable chains of thought.

To validate solutions generated by the LLM, DeepSeek avoided unreliable methods like human validation or using another LLM, which can introduce errors or hallucinations. Instead, the system asked the LLM to solve coding, math, or logic problem and compared the results to expected outputs. In the case of code generation, the code was executed and it's output compared to the ground truth. This straightforward method ensured large scale of training examples and reliable evaluation.

Let's see how R1 works in more detail.

Traditional LLM Training Pipeline

If you want to learn the fundamentals of language models and the modern machine learning, check my The Hundred-Page Language Models Book (<https://thelmbok.com>) published January 2025.

Pre-training Phase

LLM training from scratch begins with pre-training a **decoder Transformer** model on vast corpora of text data, where the model learns most of its knowledge and skills. During this phase, the model processes collections of documents consisting of trillions of **tokens** (equivalents of words or word pieces), developing an implicit understanding of grammar, factual knowledge, and basic logical relationships. The training objective focuses on

R1的贡献：
教会模型一步一步思考，且不依赖于大规模标注数据

R1只检查格式与最终的值

minimizing **cross-entropy** loss, essentially teaching the model to predict the most likely next word in any given sequence. This process creates rich internal representations of language, though the model at this stage lacks the ability to deliberately apply this knowledge to specific tasks.

The pre-training phase usually processes web text, books, scientific papers, and code repositories—essentially anything available online. This often includes data without strict regard for copyright, as the legal status of training LLMs remains unclear. More data generally leads to better models.

Supervised Finetuning (SFT) 指令遵循以及学习格式

The **supervised finetuning** (SFT) phase transforms a pre-trained model into one that can follow explicit instructions and generate appropriate responses. This process involves training on carefully curated datasets of instruction-response (or problem-solution) pairs, where each example demonstrates the desired model's behavior for a particular type of request. The training data typically includes tasks (instructions, problems) like question answering, summarization, code generation, and creative writing, with each example showing both the input prompt and the ideal response.

During SFT, the model learns to recognize instruction patterns and generate responses that align with human expectations. This phase is crucial for developing the model's ability to understand user intent and produce contextually appropriate outputs. The training data quality becomes increasingly important, as the model will emulate the patterns and styles present in the training examples. Poor quality or biased training data can lead to models that exhibit undesirable behaviors or fail to **generalize** well to real-world tasks.

This is why high-quality SFT data is often seen as a competitive advantage. It is expensive to create, as it requires manual effort, and challenging to execute properly. Done right, it effectively taps into the knowledge and skills acquired during pre-training.

Preference Optimization

The final training phase, though optional, focuses on aligning the model's outputs with human preferences, such as "I prefer this response over that one," to improve perceived quality. This is typically done through **reinforcement learning from human feedback** (RLHF) or **direct preference optimization** (DPO).

In RLHF, human annotators evaluate multiple model outputs for the same input by giving each a numerical preference rating (like between 1 and 5). These ratings are then used to finetune a separate model, called the **reward model**, which predicts a reward signal (also a numeric value). This signal is used to guides the language model toward behavior deemed desirable by those annotators.

The process works like this: the LLM generates several alternative solutions to a problem. The reward model evaluates these solutions and assigns a reward score. The LLM's parameters are then updated to increase the likelihood of generating the highest-rated solution in the future.

This preference optimization phase refines subtle aspects of the model's behavior that supervised learning alone cannot address. These include avoiding harmful content, maintaining a consistent tone and persona, producing responses that are both engaging and informative, as well as adding bias or censorship.

Chain of Thought

It has been observed that finetuning an LLM to generate a **chain of thought** (CoT)—a reasoning step before presenting the solution—can improve the quality of the solutions it generates. This intermediate reasoning step helps the model produce more accurate and thoughtful responses. The user experience looks like this:

```
User: How much is 1+1?

LLM:

<think>
OK, the user wants to know how much is 1+1. Obviously, it's 2, but is this the only possible answer? What can change? Oh, maybe 1 is not decimal? In this case 1 + 1 is 10. But is this what the user really wants? It's more likely that they use the decimal numeral system, so the answer is more likely 2.
</think>

If you mean the decimal numeral system, then the answer is 2.
```

This is a simplified example. In reality, the chain of thought could be much longer depending on the complexity of the task.

A well-known example of such a "thinking" model is OpenAI's o1 model. It achieved state-of-the-art results on challenging benchmarks, including coding tasks and math olympiad problems. However, OpenAI never disclosed what happens within the <think>...</think> tags.

This lack of transparency has led to speculation that OpenAI may have developed something unique. Many assume their advantage lies in a "secret sauce" built from hand-crafted finetuning examples of complex, and expensive-to-create, chains of thought. Replicating this could be worth billions.

DeepSeek R1-Zero and R1

DeepSeek's development of R1-Zero and R1 rethinks how large language models can be trained for advanced reasoning. Their approach questions the belief that extensive supervised finetuning is essential. Instead, they show that pure reinforcement learning, applied to *automatically generated training examples* (cutting costs to zero), can produce impressive reasoning abilities when structured correctly.

R1-Zero: Pure Reinforcement Learning

R1-Zero demonstrates that sophisticated reasoning (or chain of thought) capabilities can emerge in an LLM through pure reinforcement learning without the traditional supervised finetuning stage. This model, built on the DeepSeek-V3-Base architecture with 671 billion parameters, represents a significant departure from conventional training approaches.

Architecture Details

The foundational architecture of R1-Zero consists of 61 Transformer decoder blocks (you can learn everything about the decoder Transformer for my new book (<https://thelmbok.com>)), implementing a hybrid approach to attention mechanisms. The first three decoder blocks employ dense attention (with vocabulary size of 130,000 tokens and embedding dimensionality of 7,168), providing a foundation for basic pattern recognition and contextual understanding.

The remaining decoder blocks implement a **mixture of experts** (MoE) architecture, which allows for more efficient (cheaper and faster) processing of input tokens.

A MoE architecture represents an approach to model scaling that employs multiple specialized neural networks or experts (in R1, there are 256 experts per block) alongside a learned **routing mechanism** (typically a **gating network**) that dynamically directs different inputs to the most appropriate experts. Rather than processing all tokens through the entire network, MoE models selectively activate only a subset of experts (8 in R1) for each token, with the router determining which experts to engage based on the input's characteristics. If you aren't familiar with the inner workings of MoE, you can read about it in this article and then come back to continue reading on R1-Zero training process.

Training Process

The training process begins with the DeepSeek-V3-Base model, which has already undergone pre-training on 14.8 trillion tokens to predict the next token given the context.

Group Relative Policy Optimization (GRPO) is one of DeepSeek's innovations in reinforcement learning for language models. Let's see how it works and how it's different from traditional approaches.

RLHF with PPO

RLHF traditionally relies on the algorithm called **Proximal Policy Optimization** (PPO). PPO in RLHF is a method for training language models to produce better outputs by using reward signals. This article explains how it works. If you aren't familiar with the subject, go there and then come back to continue reading about GRPO. In short, the RLHF with PPO looks like this:

Key Components:

- 1. **Policy model** (Actor) - This is the language model being trained to generate text
- 2. **Reference model** - Usually the initial SFT model, remains "frozen"
- 3. **Reward model** - Trained on human preferences to score outputs
- 4. **Value model** (Critic) - Predicts expected future rewards for **advantage** estimation 优势估计 $A(s,a) = Q(s,a) - V(s)$

The Process:

- 1. The policy model generates text outputs for given prompts
- 2. The reward model scores these outputs
- 3. The value model estimates expected rewards at each token position
- 4. PPO calculates **advantages** - how much better/worse each action (predicted token) was compared to what was expected
- 5. The policy is updated to increase the probability of actions that led to higher-than-expected rewards
- 6. A **KL penalty** (Kullback–Leibler divergence) from the reference model prevents the policy from changing too drastically

The most challenging part in RLHF is that rewards only come at the end (when the full text is evaluated), but the model needs feedback for each token it generated. The value model helps solve this by learning to predict future rewards, allowing the system to distribute credit across all decisions that led to good outcomes.

The training loop alternates between:

- 1. Collecting batches of generations and their rewards
- 2. Updating the policy using the PPO objective 实践中，如果policy在一个batch中更新太频繁会导致训练崩溃
- 3. Periodically updating the value model to better predict rewards

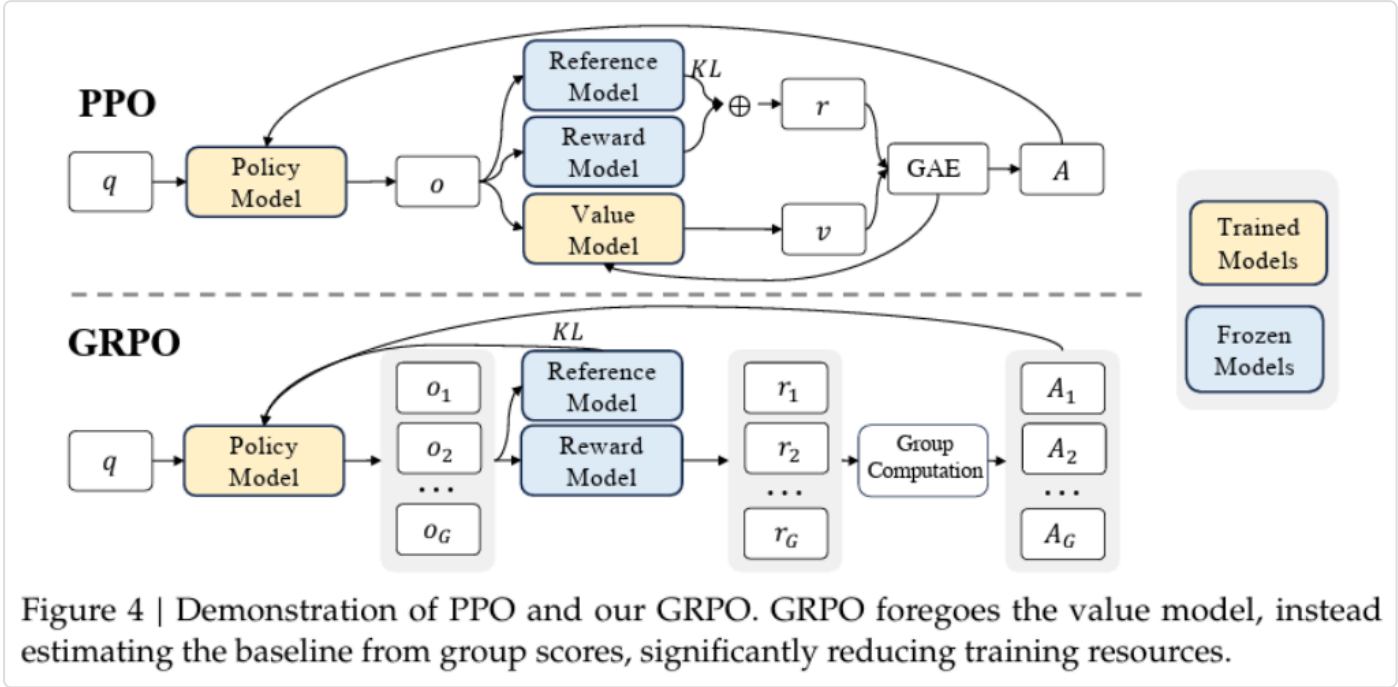
All of this works together to gradually steer the model toward generating outputs that receive higher rewards while maintaining coherent and high-quality text generation.

This is a complex optimization process that requires careful tuning of multiple components and hyperparameters to work effectively.

GRPO is a variant of PPO that eliminates the need for a value model while maintaining effective reinforcement learning. GRPO takes a different approach to PPO: instead of using a value model to estimate baselines for advantage calculation, it samples multiple outputs for each input and uses the average reward within each group as the baseline. This means if you generate 64 outputs for a math problem, and one solution gets a reward of 0.9 while the group average is 0.7, that solution would get a positive advantage of 0.2. This group-based approach provides a natural way to determine if an output is better or worse than average without needing a separate value model to make predictions. While both RLHF/PPO and GRPO aim to improve a model through reward signals, GRPO achieves this more efficiently by leveraging group statistics rather than training an additional value model. This makes GRPO different from DPO as well, since DPO focuses on directly optimizing preference probabilities between pairs of outputs rather than using any form of advantage estimation.

GRPO不需要基于NN的reward model,只需要规则的reward

而且DPO是offline的,也没有用到优势估计, ppo, grpo是online的



(./img/PPO-GRPO.png)

One of GRPO's most remarkable features is the emergence of self-verification behaviors. As training progresses, the model naturally develops the ability to catch and correct its own mistakes. This manifests in what the researchers termed "Aha moments"—instances where the model explicitly recognizes an error in its reasoning and revises its approach. For example, while solving a complex mathematical problem, the model might write "Wait, I notice an error in my previous step..." and proceed to correct its calculation. This behavior wasn't explicitly programmed but emerged as a consequence of the group-based reward structure.

自己意识到错误, 并进行改正

Reward Structure Design

The reward structure in DeepSeek's training process comprises multiple components that work together to guide the model's learning without requiring human feedback. At its core, the system uses two primary types of rewards: accuracy rewards and format rewards: 值正确与格式正确

"The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases."

"In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between <think> and </think> tags."

这个与字节的ReFT比较相似

A particularly innovative aspect of this reward structure is its ability to drive improvements without any human feedback in the loop. The system relies entirely on programmatic verification and structural analysis, making it highly scalable. This approach allows the training process to evaluate millions of responses quickly, enabling rapid iteration and improvement. The absence of human feedback also eliminates potential biases that might be introduced by human evaluators and allows the model to potentially discover novel problem-solving approaches that humans might not consider.

DeepSeek R1's Multi-Stage Training Process

Building upon the insights gained from R1-Zero, DeepSeek R1 implements a novel four-stage training pipeline designed to maintain strong reasoning capabilities while addressing the limitations of pure reinforcement learning. The specific limitations of pure reinforcement learning observed by the researchers are:

1. **Poor readability**—R1-Zero's outputs were often difficult to interpret despite being logically sound. This showed up in a few ways:
 - Fragmented or unclear explanations

而且多语言混杂

- Inconsistent formatting and structure in responses
- Difficulty in following the reasoning chain

2. **Language mixing**—R1-Zero would frequently:

- Switch between languages (particularly English and Chinese) within a single response
- Mix languages even when the query was in a single language
- Show inconsistent language use across different parts of the reasoning process

多语言混杂反而有利于
准确性

Interestingly, the researchers later, when worked on DeepSeek R1 (see below), found that enforcing single-language usage actually slightly degraded R1-Zero's performance. This suggests that the model found it more effective to reason using multiple languages, even though this made the outputs less user-friendly.

These limitations were why DeepSeek developed the full R1 model with its multi-stage training process, particularly using the **cold-start data** and additional language consistency rewards to address these specific issues while maintaining the strong reasoning capabilities.

The training process for R1 consists of four stages, each serving a specific purpose in developing the model's capabilities.

The **first stage**, known as the cold start stage, begins with a relatively small dataset:

"To collect such data, we have explored several approaches: using **few-shot prompting** with a long CoT as an example, directly prompting models to generate detailed answers with reflection and verification, gathering DeepSeek-R1-Zero outputs in a readable format, and refining the results through post-processing by human annotators."

Unlike DeepSeek-R1-Zero, to prevent the early unstable cold start phase of RL training from the base model, for DeepSeek-R1 researchers collected a small amount of long CoT examples to finetune the DeepSeek-V3-Base model as the initial RL actor. To collect such data, they have explored several approaches: using **few-shot prompting** with a long CoT as an example, directly prompting (unspecified) models to generate detailed answers with reflection and verification, gathering DeepSeek-R1-Zero outputs in a readable format, and refining the results through post-processing by human annotators.

Several thousands of such cold-start examples were collected and used to SFT DeepSeek-V3-Base.

只加了语言一致性的
奖励

During the **second stage**, the model undergoes intensive reinforcement learning focused specifically on reasoning tasks. This phase builds upon the GRPO framework used in R1-Zero but introduces additional reward components for language consistency. The training process concentrates on mathematical problem-solving, coding challenges, and logical reasoning tasks. All training examples could be generated and validated for correctness automatically:

- Math can be verified for correctness if the result is exactly as expected
- Code can be verified by executing it and comparing the output to the expected output provided, for example, by a unit test.

In this stage, and additional reward for language consistency was introduced, which help eliminate the tendency to mix languages that was observed in R1-Zero.

The **third stage** represents a significant departure from traditional training approaches. Using the partially trained model from stage two, it was further finetuned to not only succeed on math, logic, and coding tasks, but on general text generation tasks too. 引入其他领域的的文本生成任务，如写作，角色扮演

Unlike the initial cold-start data, which primarily focuses on reasoning, this third stage incorporates data from other domains to enhance the model's capabilities in writing, role-playing, and other general-purpose tasks. Specifically, researchers generated the data and finetuned the model as described below.

Reasoning data 精心策划的 curated 多语言混合的使用拒绝采样过滤掉

Researchers curated reasoning prompts and generated reasoning trajectories by performing **rejection sampling** from the second-stage-trained and using DeepSeek-V3 for judgment of its quality.

Additionally, because the model output was sometimes chaotic and difficult to read, researchers have filtered out chain-of-thought with mixed languages, long paragraphs, and code blocks. For each prompt, researchers sampled multiple responses and retain only the correct ones. In total, they collected about 600k reasoning related training examples.

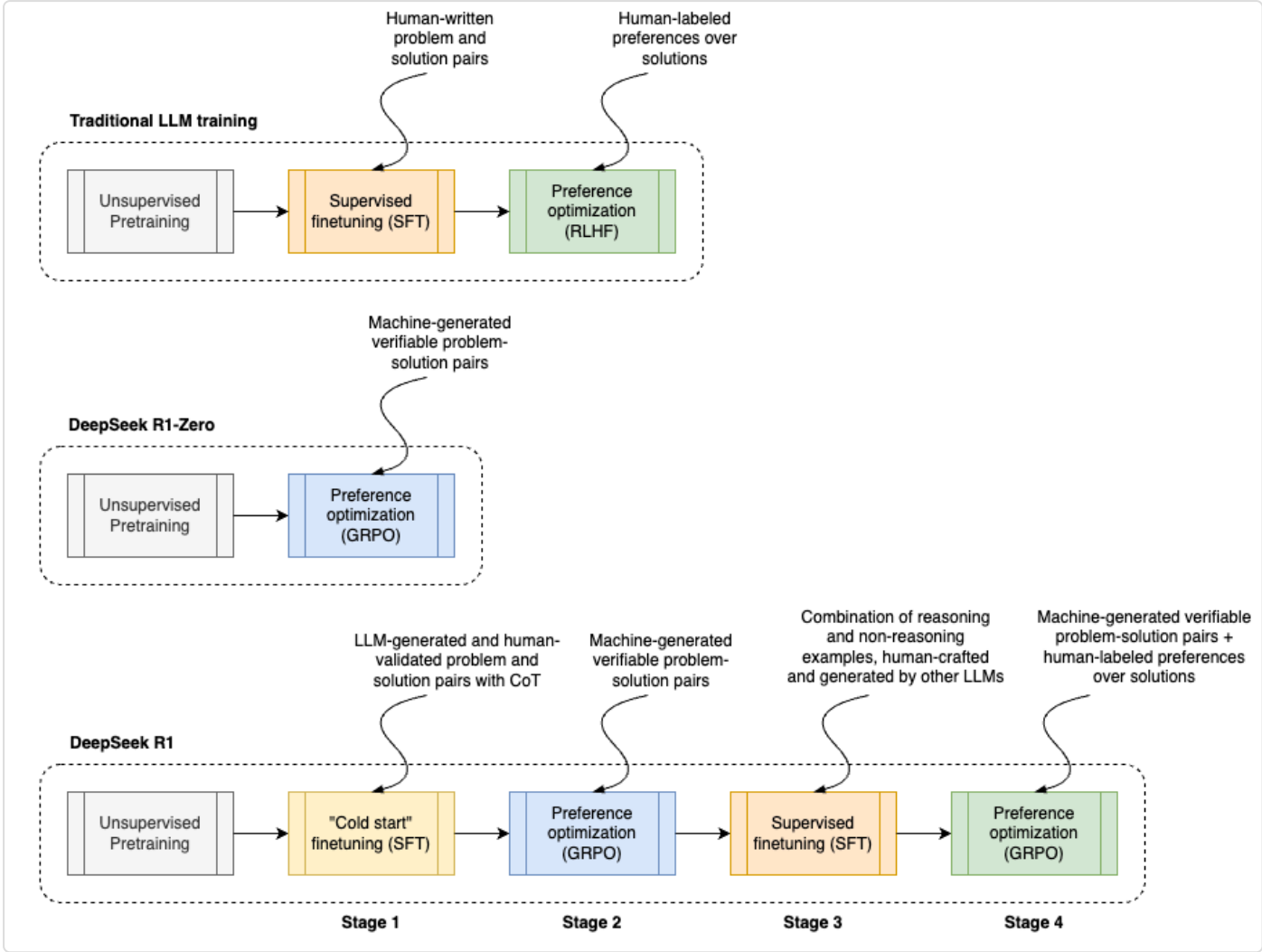
Non-Reasoning data

For non-reasoning data, such as writing, factual QA, and translation, researchers adopted the DeepSeek-V3 pipeline and reused portions of the SFT dataset of DeepSeek-V3. For certain tasks, DeepSeek-V3 was used to generate a potential chain-of-thought before answering the question. However, for simpler queries, such as "hello" not CoT was provided. A total of approximately 200k training samples unrelated to reasoning were collected. Then the stage-three model was finetuned for two epochs using the final curated dataset of about 800k examples.

The training approach at this stage is again SFT and not a continued reinforcement learning.

The fourth and final stage implements a general-purpose **preference optimization** reinforcement learning as explained in the beginning.

The three LLM training approaches compared:



(./img/DeepSeek-R1.drawio.png)

Conclusion

DeepSeek R1 represents a significant advance in AI reasoning capabilities, achieved through innovative training methodologies and architectural decisions. The success of its pure reinforcement learning approach in R1-Zero, followed by the refined multi-stage training process in R1, demonstrates that sophisticated reasoning capabilities can be developed without relying on extensive supervised training data.

The model's competitive performance against established benchmarks, combined with its cost efficiency and successful distillation to smaller architectures (will be covered in a followup article), suggests a promising direction for future AI development. As the field continues to evolve, the lessons learned from R1's development—particularly regarding the effectiveness of pure reinforcement learning and the importance of structured training stages—will likely influence the next generation of AI models.

If you want to learn the fundamentals of language models and the modern machine learning, check The Hundred-Page Language Models Book (<https://thelmbbook.com>) published January 2025.